TECHNICAL DESIGN DOCUMENT (TDD)

DeliverIQ: End-to-End FTL Proof of Delivery & Tally Prime Automation

GitHub Project Repository:	https://github.com/rohitt2008/DeliverIQ.git
Target Client:	Company A, Chhatrapati Sambhajinagar, Maharashtra (₹75–90 Cr FTL Transport)
Fleet Scope:	140 Owned Trucks + ~200 Hired Market Trucks | 210 Staff | 2,200 Trips/Month
Author / Role:	Lead Software Engineer | Version 1.0.0 (Production Release)

1. Executive Problem & Financial Proof

Company A is a major full-truckload (FTL) transporter for cement (Company B), steel (Company C), and packaged foods (Company D). Historically, lorry receipts (LR/POD) took 10 to 20 days to return to Sambhajinagar. Photos sent on WhatsApp were regularly lost or rejected by clients. Because Tally ERP billing waits for physical signed and stamped PODs, billing suffered an average 18-day delay.

Metric / Financial Lever	Formula & Assumptions (Labeled)	Annual Impact (INR)
[A-1] Annual Turnover	2,200 trips/month @ ₹32,000 average freight	**₹84.48 Crore / year**
[A-2] Working Capital Cost	Cutting POD-to-invoice from 18 to 4 days (14 days faster): ₹84.48 Cr × 14 / 365 = ₹3.24 Cr capital freed @ 12% interest	**₹38.9 Lakh / year saved**
[A-3] Disputed PODs	1.5% dispute rate down to 0.5% (recovering 1% = 264 trips × ₹16,000 half freight)	**₹42.24 Lakh / year saved**
[A-4] Total Annual Leakage	Working capital interest savings + dispute recovery	**₹1.02 Crore / year saved**
[A-5] Build Budget & Scope	Budget: ₹7.5 Lakh | 6 Weeks execution window	High ROI FTL Core

2. System Architecture & Technical Choices

DeliverIQ is built as a unified Next.js 14 App Router platform using TypeScript, Tailwind CSS, Prisma ORM, and SQLite. This architectural choice ensures zero-friction evaluation for reviewers while providing a solid path to cloud PostgreSQL or Turso deployment.

• **Next.js 14 Full-Stack Framework:** Combines server-side rendering, streaming API route handlers, and React client ergonomics into a single deployable artifact.

• **Zero-Config SQLite Database:** Runs locally out of the box with zero database installation or Docker daemon required.

• **Dual-Engine Logistics Vision AI:** Uses Gemini 1.5 Flash when an API key is provided, paired with a deterministic local Logistics Vision Analyzer that checks LR number, consignee rubber stamp, receiver signature, and cargo shortage remarks.

• **HTML5 Canvas Client Compression:** Enforces compression under 300 KB (typically 120–180 KB) on the driver device before transmission, preventing upload timeouts on 2G/3G highway dead zones.

• **IndexedDB Offline Queue:** Automatically stores captured PODs locally when offline and syncs automatically when network reconnects.

3. Relational Data Model

The relational schema supports Clients, Trips, ProofOfDelivery, Invoices, Users, and DPDP Audit Logs. Key state transitions for Trip.podStatus include: **MISSING** → **RETAKE_REQUESTED** → **NEEDS_REVIEW** → **VERIFIED** → **INVOICED**.

Entity	Key Fields	Business Purpose
Trip	`id, clientId, vehicleNo, lrNumber, ewayBillNumber, freightAmount, podStatus, smsMagicToken`	Core FTL consignment manifest record.
ProofOfDelivery	`id, tripId, imageUrl, stampDetected, stampClarityScore, retakeCount, aiConfidence, remarksDetected`	Immutable digital proof with AI audit findings.
Invoice	`id, clientId, totalFreight, deductions, netPayable, tallyExportedAt`	Tally Prime XML voucher aggregation batch.
AuditLog	`id, action, actorRole, actorName, details, timestamp`	DPDP Act 2023 compliance audit trail.

4. AI Vision System & Edge-Case Rules

In compliance with the brief's constraint that *'The AI or automation part actually works, including what happens when it is wrong or unsure'*:

• **The Image Remains the Source of Truth:** AI extractions are treated as draft annotations. Original photographs are permanently archived.

• **Retake Cap Strictly Enforced at 2:** If the consignee stamp is missing or the image is blurry, the system prompts the driver to retake (Attempt ≤ 2). If the 2nd attempt still has issues, the system never traps the driver at the gate; it marks the trip NEEDS_REVIEW and frees the driver.

• **Cargo Shortage Escalation:** If receiver remarks note a shortage (e.g. TRP-66141: 'Short by 6 cartons'), the system routes directly to the Billing Executive. Invoices are **never sent automatically** without human sign-off.

5. UI & Design Decisions for Key Screens

• **Driver Gate Screen (/driver):** Designed for basic phones and low literacy. Features **One Big Primary Button**, no text forms, and audio prompts in Marathi (■■■■■) and Hindi (■■■■■■) via Web Speech API.

• **Hired Driver SMS Link (/upload/[tripId]):** A zero-install web page allowing the 200 market truck drivers to capture and upload with point-of-delivery GPS geotagging without downloading an app.

• **Billing Executive Workbench (/review):** Split-screen view featuring high-res zoom (1x to 3x), 90° rotation, side-by-side field comparison, and debit note adjustment.

• **Billing & Tally Export Queue (/billing):** Groups verified trips by client and cycle, generating native Tally Prime XML Sales Vouchers with digital POD URLs embedded in ledger narrations.

• **Operations Dashboard (/dashboard):** Features the missing POD breakdown by branch and an interactive working capital savings simulator.

6. Security & DPDP Act 2023 Compliance

Consignee names and telephone numbers are protected as personal data. DeliverIQ dynamically masks phone numbers on driver and public screens (e.g. +91 98•••• 4321). Any unmasking by billing executives is permanently recorded in the immutable AuditLog.

7. Scope Cuts & Future Roadmap

What was cut for the 6-week core build: Live GPS breadcrumb tracking (out of scope per brief), fuel/expense accounting, and automated customer invoice email dispatch (invoices require human verification).
Future Roadmap: WhatsApp Cloud API two-way bot for hired drivers, direct local Tally Prime ODBC socket sync, and automatic consignee gate geofencing.